Nombre del Alumno:

UNIVERSIDAD COMPLUTENSE MADRID

Examen de Sistemas Inteligentes

20 de mayo de 2025

Instrucciones generales para realizar el examen:

- **Contesta a las preguntas digitalmente usando Word**. Puedes apoyarte en tantas hojas en sucio adicionales como quieras, pero las respuestas deben ir en el examen de forma digital.
- **El examen escrito se entrega en formato PDF para evitar modificaciones**. Para guardar el formato pdf ir a Archivo>exportar>crear pdf. **NO SE ACEPTAN ENTREGAS QUE NO ESTEN EN FORMATO PDF.**
- El tiempo para la realización del examen es de **3h**. Consta de dos partes, una teórica con problemas que se contestan en el Word de 5 puntos y una práctica que se contesta usando Júpiter Notebook. La entrega debe contener un zip con lo siguiente en la raíz del zip: El pdf con las respuestas de la parte teórica, el notebook con la implementación de la parte práctica y los datos que se necesiten (dataset). En Jupiter Notebook podéis escribir cualquier consideración de ejecución de la práctica, pero es requisito obligatorio que **la parte práctica sea ejecutable sin la intervención del profesor**.
- Haz la entrega en el campus con tu cuenta de la UCM. Por seguridad, **Pon tu nombre al zip para evitar problemas y pon también tu nombre en el documento pdf y en Júpiter notebook**.

Preguntas cortas de teoría. (2,5 p)

Contesta a las siguientes preguntas cortas de teoría. Cada pregunta vale 0.5 puntos.

1. Amazon ha diseñado un sistema automatizado en su almacén donde un conjunto de robots heterogéneos buscan realizar lo más rápido posible los envíos que se solicitan. Existen robots que permiten conseguir los paquetes que se sitúan en lo más alto. Otros robots que se encargan de manipular los paquetes almacenados hasta 1 metro del suelo y luego existen otros robots que son capaces de recoger todos los paquetes y enviarlos a sus destinatarios. Todos los robots son eléctricos y necesitan recargar sus baterías. Para ello existen algunos cargadores

diseminados por la nave. ¿Qué tipo de sistema de agentes dispone Amazon? Razona la respuesta.

Respuesta: Los agentes que comparten un objetivo común, que es enviar los paquetes lo antes posible. No tienen habilidad suficiente, por lo que deben de coordinarse. No hay recursos suficientes debido a que deben compartir los enchufes de carga de la nave. => Colaboración coordinada.

2. Tenemos un agente que es capaz de colocar una colección de libros (10 por ejemplo) que deben ser colocados en una estantería compuesta por distintos estantes (por ejemplo 3 estantes). Cada libro tiene una altura determinada, mientras que los estantes disponen cada uno de una altura diferente y una capacidad diferente (número de libros que caben). Determinar una distribución de libros en los estantes de forma que se maximice el número de libros colocados (no es necesario ubicar todos los libros en la estantería). ¿Cómo implementarías el agente? Modélalo.

Respuesta: Es un claro ejemplo de un problema resoluble con métodos de búsqueda local y optimización como por ejemplo genéticos. Si implementamos con genéticos, el cromosoma sería los 10 libros que disponemos, 0 seria no colocado, 1 sería colocado en la estantería 1, 2 en la estantería 2 y 3 en la estantería 3. Como tenemos una altura diferente en cada libro, habrá libros que no quepan en un estante. El fitness puede ser el número de libros colocados correctamente. Para ello contamos los libros que caben en cada estantería y vemos si hay más libros de los permitidos. también vemos si alguno de los libros no cabe en altura.

Se podría con A* para codificarlo la solución sería el orden en el que se colocarían los libros. Es decir, si tenemos 10 libros (L1, L2, …. L10) y 3 estanterías (E1,E2,E3) una posible solución (asumiendo que las restricciones se cumplan) sería: L1-E1, L2-E1, L3-E1, L4-E1, L5-E2, L6-E2, L7-E2, L8-E3, L9-E3, L10-E3.

Así que el nodo sería, igual que antes un vector que representa los libros de enteros, cuyo valor representa la estantería a la que va. 0 implica que no hay ninguno colocado. Cada estantería tendrá un tamaño en alto y un número de libros que admite. Estos son datos estáticos (no cambian con cada nodo)

Estado inicial (0,0,0…,0) que indica que no hay ninguno establecido.

Estado final: todos los libros han sido colocados (no hay ningún cero)

Heurística: numero de ceros que faltan por cambiar, que es equivalente a número de libros por colocar. En verdad no ayuda demasiado ya que todos los nodos potencialmente tendrían el mismo peso. Habría que buscar una heurística más informada para que no fuese prácticamente búsqueda en anchura.

Operadores: Colocar(Libro,Estante): hay que asegurarse de que, el libro elegido cabe en altura en el estante elegido y que el número de libros almacenados en cada estante es menor que el máximo de cada estante. Si esto llega a ocurrir, el operador no se completaría.

Como se puede ver el factor de ramificación es enorme, en cada nodo, potencialmente podemos poner todos los libros en todos los estantes con lo que el sucesor del nodo inicial puede contener

un gran numero de posibles combinaciones. El proceso de búsqueda es casi a ciegas incluso con una buena heurística, es por eso que en este tipo de problemas suele funciona mejor algoritmos subóptimos como genéticos y **es preferible elegirlos para resolver el problema**.

3. Estamos diseñando un sistema de navegación que permita a un dron, moverse por un espacio tridimensional donde X es la anchura, Y la altura y Z la profundidad. El espacio tridimensional está discretizado usando cubos que denominamos voxels. Como si de Minecraft se tratase, los objetos pueden ocupar una serie de voxels. Un Voxel se considera ocupado si al menos dentro de su volumen hay una parte del objeto ocupado. El dron es capaz de volar en todas las direcciones, pero es más rápido moviéndose sobre su mismo plano. Si se mueve sobre su plano, es capaz de moverse a velocidad 2. Pero si cambia de plano (se mueve hacia arriba o hacia abajo) lo hace a velocidad 1. El dron no es capaz de avanzar o retroceder y ascender, es decir que puede moverse en cualquier dirección en el plano en el que se encuentre, pero no puede ascender o descender a la vez. Cuando asciende o desciendo, lo hace en el sitio sin avanzar. Sabiendo esto. ¿Cuál de estas heurísticas sería más adecuada para A* si quiero ? Justifica la respuesta.

a) Distancia: $d(A,B) = \sqrt{((xa - xb)^2 + (ya - yb)^2 + (za - zb)^2)}$

b) Distancia: $d(A,B) = \sqrt{(0.5 * (xa - xb)^2 + (ya - yb)^2 + 0.5 * (za - zb)^2)}$

c) Distancia: $d(A,B) = \sqrt{0.5 * (xa - xb)^2 + 0.5 * (za - zb)^2} + |ya - yb|$

d) Distancia: $d(A,B) = 2 * |xa - xb| + |ya - yb| + 2 * |za - zb|$

Respuesta: la d no es admisible, la mejor es la C porque es la que más se ajusta al problema ya que el movimiento en Y es por manhattan mientras que el de X y Z es por euclídea. Además, como en X y Z se mueve a velocidad 2, se mueve más rápido que en Y que se mueve a velocidad 1, de ahí que haya que multiplicar por 0.5 porque tarda menos en llegar al destino.

4. Elige a priori el mejor modelo de Machine Learning para un problema de clasificación binaria que pretende predecir si los alumnos de una clase van a aprobar una asignatura. Disponemos de 5000 datos de entrada con información de: Peso, Altura, Genero (masculino, femenino u otro), Renta familiar del año anterior, horas que dedica a videojuegos, horas que dedica lectura, nivel de actividad física (baja, media o alta) y la nota del curso anterior. Sabemos que algunos datos de los que disponemos pueden no estar correctos, ya que algunos profesores del centro donde hemos extraído los datos eran un poco despistados y nos consta que ha habido ciertas reclamaciones de errores en las notas. Una vez elegido el modelo, escribe brevemente las entradas y salidas que tendría el modelo, que atributos utilizarías o cuales descartarías y cualquier otra consideración que estimes oportuno para construir el modelo.

Respuesta: El peso y la altura a priori no sirven para lo que se está buscando, se eliminarían. El género habría que hacer OHE y saldrían 3 entradas. El nivel de actividad física puede ser codificado con label enconding y dejarlo en 1 campo ordinal. El resto son valores numéricos. Como tenemos sospechas de que hay ruido en la entrada, vamos a utilizar a priori un multi layer perceptrón con 8 entradas y una salida. Los datos estarán normalizados.



5. Estamos procesando datos sobre videos musicales en formato sin comprimir, es decir, el video está almacenado frame a frame. Disponemos de multitud de videos de diferente longitud y con diferente resolución descargados de YouTube y queremos construir un modelo que nos diga qué tipo de música estamos escuchando. Los tipos de música que pretendemos clasificar son: Rock, Pop, Techno, Clásica y Reguetón. Describe brevemente que modelo o conjunto de modelos utilizarías, que transformación de datos necesitaría si fuese el caso y por qué lo has elegido.

Respuesta: lo primero que debemos hacer es normalizar la entrada a una resolución y una longitud concreta y procesar los videos dejándolos a dicha resolución y cortándolos a dicha longitud. Si después de dividir el video en partes alguna de estas es menor que la longitud determinada, podemos descartarla. Después usaremos o bien un modelo convolucional o bien un modelo de Autoencoder como Extractor de características. La idea es reducir la dimensionalidad del problema todo lo que podamos. Después usaríamos una capa densa o un perceptrón multicapa tradicional para la capa de clasificación. Valdría también una convolucional y un LSTM para pillar la parte recurrente (o cualquier recurrente) incluso estaría bien cualquier otro clasificador a continuación del autoencoder o la CNN ya que es un ejercicio teórico.

Problema a resolver (2,5 p)

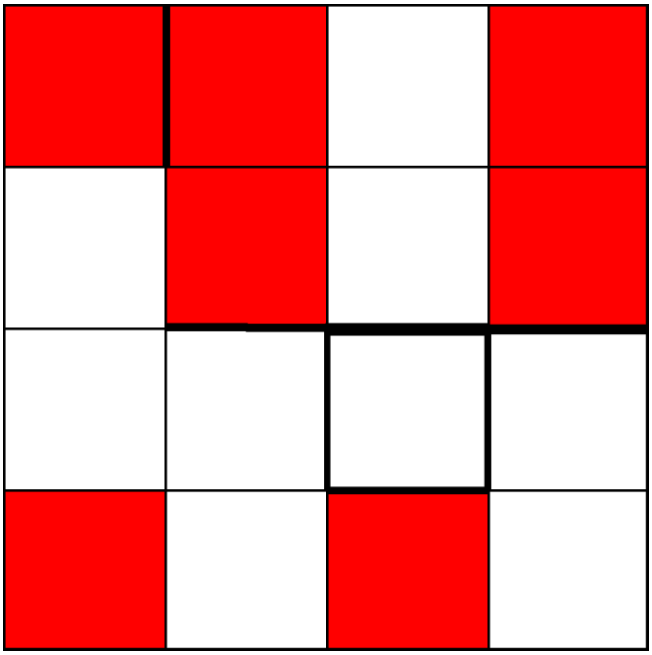
Queremos desarrollar un agente que se encarga de resolver puzles del videojuego El Profesor Layton. Para el que no los conozca, es una serie de videojuegos originarios de la videoconsola Nintendo DS que han ido teniendo diferentes entregas en múltiples consolas (Nintendo 3Ds, Switch) y móviles. La idea es que los desarrolladores del juego quieren convertirlo en un juego multijugador, donde se compita por ver quien es el jugador más rápido en resolver un puzle. Como es posible que, al principio, no haya suficientes jugadores humanos, los desarrolladores han optado por crear un agente que sea capaz de resolver los puzles de forma automática. En concreto y como prueba de concepto para ver su viabilidad, quieren diseñar un agente que resuelva el puzle de "Pisotea la Tarima" (*Stomp on It!) que aparece en el videojuego: El profesor Layton vs. Phoenix Wright: Ace Attorney.*



El puzle consiste en saltar sobre las diferentes baldosas del juego para conseguir que se complete correctamente el dibujo. Se comienza con el tablero con esta configuración:

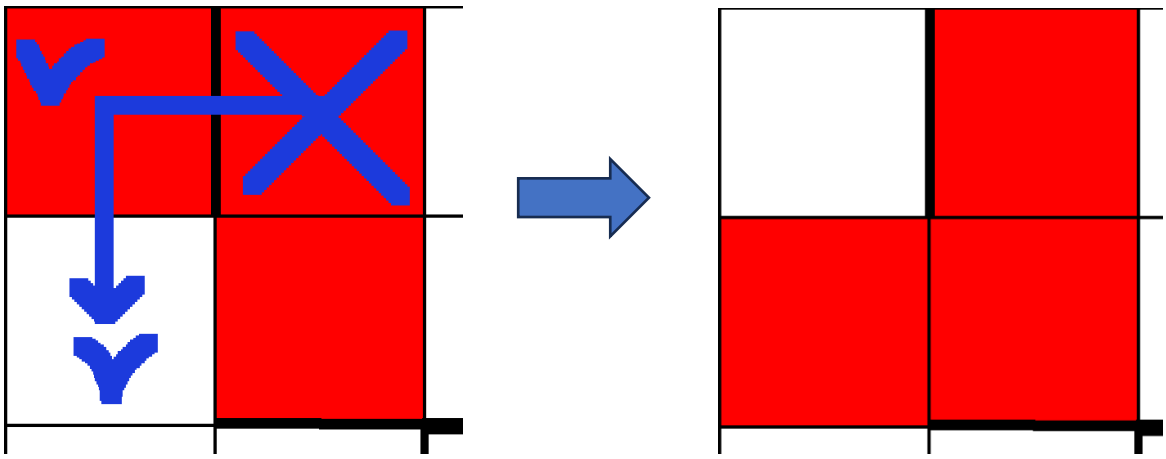


Y debe acabar como indica la primera figura. Podemos simplificar el problema a un problema donde las casillas tiene dos caras una cara A (roja) y una cara B (blanca)



Y el problema consiste en cambiar todas las caras al mismo color.

Cada vez que saltes sobre una baldosa, se gira la baldosa y las colindantes (arriba, abajo, izquierda, derecha) pero si entre la baldosa y la adyacente hay un muro (un marco negro más grueso), la adyacente no se volteará.



El agente tarda 1 turno en moverse en una de las 4 direcciones posible, de una casilla a otra si no hay barrera y 2 turnos si hay barrera (puede saltarla). Mientras que tarda 2 turnos en saltar sobre una casilla y que ésta y sus adyacentes giren.

En el juego multijugador, cada jugador resuelve el puzle de forma independiente y el que lo haga en menos pasos ganará. Si se hace en el mismo número de pasos, hay empate y ambos

ganan. Así pues, los diseñadores han planteado que exista dos agentes, uno para el nivel fácil (Que llamaremos agente **Easy**), donde no es necesario obtener el mínimo número de pasos y otro de alto nivel (que llamaremos **Hard**) donde el agente debe resolver el problema en el menor número de pasos.

Se pide:

1. ¿Qué tipo de agente es Easy y que tipo de agente es Hard? Razona la respuesta. (0.5p)

Respuesta: Ambos son agentes deliberativos basados en objetivos, solo que el primero puede ser resuelto por un algoritmo no completo como por ejemplo búsqueda voraz, o con genéticos, mientras que el otro si es posible sería resolverlo con A*.

2. Modela uno de ellos usando REAS (0.5p)

Respuesta:

Rendimiento: número de pasos en resolver el puzzle

Entorno: tablero 4X4, baldosas con dos caras, paredes que impiden moverse a las baldosas colindantes, un agente que se mueve y que salta sobre las baldosas. El entorno es determinista, secuencial, de información completa, etc.

Actuadores: moverse en 4 direcciones, y saltar.

Sensores: el agente no dispone de sensores interesantes más allá de saber en qué casilla está. Dispone de información de todas las casillas y todas las posiciones.

3. Modela el agente Hard con el formalismo que consideres más adecuado (1p).

Respuesta:

El agente Hard se modela con A*. Estado inicial el indicado en la figura. Estado final con todo en rojo.

El agente internamente tiene una matriz de 4X4 u una lista de posibles restricciones de propagación del salto. Además, tiene la posición X,Y del agente en el mundo.

Operadores:

- Moverse a la izquierda / derecha / arriba / abajo => coste 1 Cambia la posición del agente a donde corresponda salvo moverse a través de pared que serían 2.
- Saltar, coste 2. Al saltar no todos los nodos, modifica los valores adyacentes al jugador siempre que se pueda.

Heurística:

Más básica: número de baldosas de tipo B (blancas) Cuantas menos haya, más cerca estaremos de resolver el puzle. Pero ojo porque no es admisible ya que podríamos quitar de un salto hasta un total de 5 casillas. Si solo tenemos por ejemplo 4 casillas que cambiar y las cambiamos de 1 salto, entonces solo hemos consumido 2 turnos no 4. Así que no nos vale. Deberíamos dividir el número de casillas blancas libres entre el máximo número de casillas que podemos

cambiar. En este caso son 5 los que podemos cambiar como máximo, X 2 porque tardas 2 turnos.

Una mejora de esta sería tener en cuenta la distancia de manhattan desde donde nos encontramos, a la casilla más alejada blanca del tablero -1. Está claro que por muy bien que lo hagamos si el muñeco está en la esquina superior izquierda y la esquina inferior derecha está en blanco, tendré que desplazar el personaje mínimo, mínimo, hasta una de las adyacentes. No podemos hacerlo para cada casilla, porque como no sabemos cuales se darán la vuelta no sabemos las que cambiaremos o no. Pero al menos sabemos que tendremos que ir hasta la más alejada.

4. Si el agente y el jugador jugasen en el mismo tablero y cada uno tuviera un turno para mover, de forma que cada uno tuviese que hacer un dibujo diferente (es decir, el jugador tuviese que voltear todas las baldosas a la cara A y el agente todas las baldosas a la cara B) ¿Qué tipo de agente sería y con que método podrías desarrollar el agente? (No es necesario modelarlo). Razona la respuesta. (0.5p)

Respuesta: En este caso el sistema multiagente es un sistema de competición individual y de suma cero, mis movimientos favorables son desfavorables para el adversario. Haríamos busca con adversarios usando Mini Max o poda Alpha Beta.

Ejercicio práctico: (5 puntos)

Somos un grupo de recién graduados en Ciencias de la Computación y hemos montado una pequeña empresa de análisis de datos. Un aeropuerto comarcal nos contrata para construir un modelo que prediga la satisfacción de los usuarios de su aeropuerto. Para ello, nos proporcionan un histórico de datos de satisfacción de sus clientes. Nos piden crear un modelo de Machine Learning que sea capaz de predecir, principalmente cuando va a estar un usuario descontento con el servicio prestado. De esta forma, de forma temprana, los miembros del aeropuerto intentarán tomar medidas para evitar que el usuario acabe valorando el servicio negativamente.

Se pide:

1. Procesa el datase Train.csv en un Jupiter Notebook y limpia el dataset de errores. Al menos debe haber un campo que este imputado el resto a libre criterio del alumno. 0.5p
2. Muestra en una gráfica 2D de nube de puntos, como está distribuida las dos clases y en base a lo que se ve, di de forma razonada si crees que al menos a simple vista parece que las clases sean linealmente separables o no. 0.5p
3. Di cual es el atributo que a priori mejor clasifica o está más directamente relacionado con la clase 0.5p
4. Crea un modelo usando el perceptrón multicapa de sklearn que consiga un rendimiento en accuracy del 95% o superior. Muestra sus matrices de confusión. 1p
5. Crea un modelo usando Random Forest que consiga un 95% o superior. Muestra sus matrices de confusión. 1p
6. Elige un tercer modelo, el que creas es más prometedor, e intenta sacar el mejor resultado posible. 1p
7. En base a los resultados de los tres modelos, di con cual te quedas, razonando la respuesta. 0.5p